# MCPBridge — An Agent That Discovers & Uses Tools Automatically

> **Short description:** A single ReAct agent whose tools are **not** written into its own code. It connects to one or more live MCP servers at startup, discovers whatever tools they expose, and calls them through the protocol — so when a server adds or changes a tool, the agent picks it up with no code change.

**Tech:** Python · `mcp` (official MCP SDK) · OpenAI SDK (function calling) · asyncio · Pydantic · **Type:** Single agent (ReAct over discovered MCP tools) · **Level:** Intermediate–Advanced

## Overview

Every other agent in this curriculum has its tools written directly into its own code — a `@tool` function sitting in `tools.py` that the loop calls. MCPBridge doesn't. It connects to live **MCP servers**, asks each one *"what tools have you got?"*, and uses whatever comes back — the same think→act→observe loop, pointed at a **discovered** tool list instead of a written one.

- **Discovers, never hardcodes** — the tool list comes entirely from `list_tools()` at runtime.
- **Many servers at once** — tool names are namespaced (`notes__search_notes`) so two servers can expose the same name without colliding, and every call routes back to the right server.
- **Degrades, never crashes** — an unreachable or mid-task-dropped server is logged and skipped; the agent keeps working on whatever tools remain.

This is the **consumer** side of a real tool protocol — the direct companion to Project 7 (which builds an MCP *server*). Point MCPBridge at Project 7's server, at the example servers shipped here, or at any conforming MCP server.

## Why

Real tools often live somewhere the agent's own codebase can't reach — a teammate's notes server, a shared company search service, a public MCP server. Hardcoding a copy of each one means every change the tool's owner makes has to be copied by hand, everywhere. Discovering tools at runtime means the agent always matches what the server actually offers *right now* — which matters most exactly when the agent and the tool provider ship on different schedules.

## Features

- **Runtime discovery** over stdio with `list_tools()` — no tool written into the agent's code.
- **Hand-built ReAct loop** (no `create_agent` / `AgentExecutor`) that picks and calls a tool it never knew about at write-time.
- **Schema bridge** — converts each MCP tool's `input_schema` into the shape OpenAI's function-calling expects.
- **Multi-server pool** — connects to every configured server, namespaces tool names, routes each call to the right server.
- **Resilience** — a per-server connection timeout; an unreachable server is skipped, a tool call that fails mid-task becomes a clear observation fed back to the model.
- **Bounded** — a hard `max_iterations` cap, raised as a named `MaxIterationsExceeded` error.

## Demo

```
$ python main.py
Connected servers: ['notes', 'filesystem']
Tools discovered: 4

Task: Check my notes for anything about the deploy process, and also list
the files in the project folder.
  -> notes__search_notes {'query': 'deploy'}
  -> filesystem__list_files {'directory': 'project'}
Answer: Your notes say the deploy process runs deploy.sh after the staging
check; the project folder contains a deploy.sh file.
```

With the `notes` server deliberately not started, the same run prints `Connected servers: ['filesystem']` and the agent answers with only the filesystem tools instead of crashing.

## Architecture

```
your task
   │
   ▼
run_agent ──► ask model (task + discovered tool schemas)
   │                  │
   │                  ▼
   │          tool_calls?  ──no──►  final answer
   │                  │ yes
   │                  ▼
   │          MCPServerPool.call_tool("notes__search_notes", {...})
```

```
    │                 │   (routing table → right server's session)
    │                 ▼
    │          observation (CallToolResult.content[0].text)
    └──────────────◄──┘   (appended, loop repeats, bounded by max_iterations)

startup:  MCPServerPool.connect_all()
          → for each server: stdio_client + ClientSession + initialize()
          → list_tools() → namespace as "servername__toolname"
          → unreachable server: logged and skipped, not fatal
```

- **One place owns the servers** — `MCPServerPool` connects, discovers, namespaces, and routes. The loop only ever sees a flat list of tool names.
- **The loop never changes** — it's the same tool-calling loop as any hand-built agent; only *where the tools come from* is different.

## Project Structure

```
<repo folder>/   (MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically/)
├── servers/
│   ├── files_server.py         example MCP server (files) for local testing
│   └── notes_server.py         example MCP server (notes) for local testing
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py              create_client(), ask_model()
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py           get_logger() — logs to stderr
│   ├── __init__.py
│   ├── agent.py               run_agent(): the ReAct loop, AgentResult
│   ├── config.py              Config, load_config()
│   ├── exceptions.py          MissingConfigError, MaxIterationsExceeded, …
│   ├── server_pool.py         MCPServerPool: connect, discover, route
│   ├── servers_config.py      SERVERS = [{name, command, args}, …]
│   └── tool_bridge.py         mcp_tools_to_openai_schema(), extract_text()
├── tests/
│   ├── integration/
│   │   └── test_agent.py      real servers: all four cases
│   └── unit/
│       ├── test_config.py     no server, no network
│       └── test_tool_bridge.py
├── .env                        your real key — never committed
├── .env.example                the setting names, no real values
├── .gitignore
├── LICENSE
```

```
├── main.py                      the demo you run
├── README.md
└── requirements.txt
```

## Getting Started

**Requirements:** Python 3.10+, an OpenAI API key, and at least one MCP server to connect to (the example servers in `servers/` work out of the box).

```
git clone <this-repo-url>
cd MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # then put your key in .env
```

`requirements.txt`: `mcp`, `openai`, `python-dotenv`, `pydantic`.

## Configuration

| Variable | Required | Default | Meaning |
|---|---|---|---|
| `OPENAI_API_KEY` | yes | — | Your OpenAI API key |
| `LOG_LEVEL` | no | `INFO` | `DEBUG`, `INFO`, `WARNING` or `ERROR` |
| `MCP_CONNECT_TIMEOUT` | no | `10` | Seconds to wait for a server to finish starting |
| `MCP_MAX_ITERATIONS` | no | `6` | Hard cap on ReAct loop steps |

## Usage

```
python main.py
```

The demo connects to the servers in `src/servers_config.py`, discovers their tools, runs a task that needs both, and prints each tool call and the final answer. Point it at your own servers by editing `src/servers_config.py`.

## Running the Tests

```
# unit tests — no server, no network, no API key
python -m tests.unit.test_config
python -m tests.unit.test_tool_bridge

# integration test — launches the example servers over a real MCP connection
python -m tests.integration.test_agent
```

## Error Handling

| Situation | What happens |
|---|---|
| `OPENAI_API_KEY` missing | Stops at startup with one clear line |
| A server never finishes starting | `initialize()` times out; that server is logged and skipped |
| A server's launch command is wrong | The connection raises; caught, logged, skipped — others still connect |
| A tool call fails mid-task | A clear `Error: …` string is fed back to the model as the observation |
| The model loops without finishing | `MaxIterationsExceeded` after the hard cap, instead of forever |

## Design Decisions

- **Namespacing uses `__`, not `.`** — OpenAI's function-name field allows only letters, digits, `_` and `-`, so a dotted `notes.search_notes` is invalid. The routing table maps `notes__search_notes` back to `("notes", "search_notes")`.
- **The whole client is async** — the `mcp` SDK's client is async-only; `main.py` wraps everything in `asyncio.run()`.
- **Context managers are entered in the main task** — never inside `asyncio.wait_for`, which runs its coroutine in a new task and would make teardown fail ("cancel scope in a different task"). Only `initialize()` is wrapped in the timeout.

## Limitations and Roadmap

- Example servers keep data in memory; swap in real servers (Project 7, the reference filesystem server, etc.) for anything real.
- stdio transport only; the same pool shape extends to HTTP/SSE transports.
- One agent. Multiple agents each with their own server pool is Project 11's territory.

## License

MIT — see `LICENSE`.